From Heuristics to Graph Neural Networks: A Systematic Experimental Benchmark of Modularity-Based Graph Clustering

Adil Chhabra[1], Kenneth Langedal[1,*], Christian Schulz[1]

[1]Heidelberg University, Heidelberg, Germany

[*]Corresponding author: kenneth.langedal@informatik.uni-heidelberg.de

Abstract

Modularity maximization is one of the most widely used objective functions for community detection in networks, balancing intra-community connectivity against a random null model. Over two decades, algorithmic development has split into two largely isolated research communities: classical discrete optimization heuristics operating on topology alone, and deep learning architectures, primarily graph neural networks (GNNs), that integrate node attributes into continuous modularity relaxations. Because each line of work evaluates almost exclusively against baselines within its own paradigm, practical trade-offs among optimization quality, scalability, and hyperparameter sensitivity remain unexplored. Here, we present a systematic experimental benchmark evaluating 17 representative graph clustering algorithms, centered on modularity maximization alongside key non-modularity reference baselines across seven algorithmic paradigms (greedy agglomerative, multilevel, memetic, streaming, flow/spectral, exact integer linear programming, and neural/GNNs). We evaluate methods across real-world attributed networks, large-scale pure topology benchmarks up to 3.3 billion edges, and synthetic graphs under controlled topological and attribute signal degradation. Across three standardized evaluation protocols isolating out-of-the-box defaults, hardware acceleration, and hyperparameter tuning, our findings reveal clear trade-offs: classical heuristics (notably VieClus and Leiden) dominate pure modularity and scalability without parameter tuning, while GNN-based methods excel in ground-truth community recovery when informative node attributes exist. However, GNNs require heavy hyperparameter tuning, have lower memory ceilings due to accelerator VRAM constraints, and suffer substantial performance drops as attributes degrade. We additionally contribute a unified, open-source framework for interfacing all the evaluated clustering algorithms, supporting reproducibility and future benchmarking.

1 Introduction

Networks are useful in modeling complex systems across the natural sciences, social sciences, and engineering. Such systems are represented as graphs, where vertices denote entities and edges encode interactions between them, as in social networks, communication infrastructures, biological and financial systems, and information networks. A common feature of real-world graphs is *community structure*, where vertices form groups with dense internal connectivity and sparser external connections.

Identifying these groups, commonly termed *graph clustering* or *community detection*, is a central task in network analysis, with applications ranging from social and political analysis and fraud detection [1] to bioinformatics [2,3], program optimization [4,5], and epidemic modeling [6]. Beyond application-specific insights, community structure reveals fundamental organizational patterns, such as cohesive subgraphs, boundary vertices, and inter-community interactions. Despite extensive study, no single clustering method performs best across all network types [7], motivating continued algorithmic development and comparative evaluation.

Most community-detection methods formulate clustering as an optimization problem over vertex partitions, assigning a quality score to each candidate solution. Among clustering objectives, *modularity* [8,9], which measures the density of edges inside communities as compared to edges between communities while accounting for random chance, is widely adopted and shown to be effective [10,11]. Thus, this benchmark focuses on modularity-based methods, while including widely used non-modularity baselines.

We distinguish two broad classes of modularity-based algorithms. *Classical* methods rely on discrete optimization heuristics driven solely by graph topology and a well-defined objective function, yielding interpretable optimization behavior. In contrast, *learning-based* approaches integrate modularity into deep learning pipelines, most commonly via graph neural networks, incorporating node attributes alongside graph topology through feature representations. This added expressiveness introduces trade-offs in interpretability, training complexity, and computational cost [12].

Community detection faces challenges arising from the scale, complexity and frequently-updating content of real-world graphs. Effective methods must balance solution quality, often with noisy data lacking ground truth, with runtime and memory scalability. Sensitivity to parameters, such as assumed cluster counts, further complicates practical deployment [13]. Understanding how different algorithms navigate these trade-offs remains an open challenge.

Despite existing surveys [13–19], there is no comprehensive experimental study that jointly evaluates classical and learning-based modularity optimization methods under a unified framework. This paper addresses this gap with the following:

Contributions

- We present the first large-scale experimental benchmark that jointly evaluates classical heuristics and modern learning-based algorithms across 17 representative modularity-based clustering methods.
- We release a unified open-source experimental framework that adapts both classical and learning-based methods behind a common interface, enabling reproducible benchmarking and future research.
- We establish a conceptual taxonomy and comparative survey organizing modularity-based algorithms across seven optimization paradigms, detailing how classical discrete heuristics and deep GNN architectures formulate and optimize modularity objectives.
- We provide comprehensive empirical insights across three standardized evaluation protocols (default settings, hardware acceleration, and per-instance hyperparameter tuning) evaluated on real-world attributed graphs, large-scale benchmarks up to 3.3 billion edges, and synthetic graphs under controlled signal degradation. Our findings uncover significant

trade-offs: classical heuristics (VieClus and Leiden) dominate pure modularity and scalability without tuning, whereas GNNs excel when attributes align with ground truth but incur a steep hyperparameter tuning tax and strict accelerator VRAM capacity limits.

2 Preliminary

Graphs and clusterings. An *(attributed) graph* is defined as $G = (V, E, \mathcal{F})$, where $V = \{v_1, v_2, \ldots, v_n\}$ is a set of n vertices, $E \subseteq V \times V$ is the set of undirected edges, and $\mathcal{F} = \{f_1, f_2, \ldots, f_n\}$ denotes node attributes, with each $f_i \in \mathbb{R}^x$ an attribute vector associated with vertex v_i, and x the attribute dimension. We write $n := |V|$ and $m := |E|$ for the number of vertices and edges, respectively. For a vertex $u \in V$, its *neighborhood* is $N(u) := \{v \in V : \{u, v\} \in E\}$, and its *degree* is $\deg(u) := |N(u)|$. Let $A \in \{0, 1\}^{n \times n}$ denote the adjacency matrix of G, where $A_{ij} = 1$ if $\{v_i, v_j\} \in E$ and $A_{ij} = 0$ otherwise. Let $\mathbf{d} \in \mathbb{R}^n$ denote the degree vector with $\mathbf{d}_i = \deg(v_i)$.

A *clustering* C of G is a partition of the vertex set V into k disjoint blocks $V_1, \ldots, V_k$ such that $V_1 \cup \cdots \cup V_k = V$. Classical algorithms operate solely on the graph structure (V, E), while learning-based methods additionally exploit node attributes $\mathcal{F}$. Learning-based approaches often aim to discover communities that are coherent both in terms of graph connectivity and attribute similarity.

Modularity. The *modularity* score of a clustering is the fraction of edges that connect two vertices within one cluster, minus the expected fraction if the edges were distributed at random. Let $L_i := |\{\{u, v\} \in E : u, v \in V_i\}|$ denote the number of edges internal to block V_i. The modularity of C is defined as

$$Q(C) := \sum_{i=1}^{k} \left(\frac{L_i}{m} - \left(\sum_{u \in V_i} \frac{\deg(u)}{2m} \right)^2 \right)$$

Equivalently, modularity can be expressed in spectral form. Let $C \in \{0, 1\}^{n \times k}$ denote the (hard) cluster assignment matrix, where $C_{ip} = 1$ if vertex v_i belongs to cluster p and $C_{ip} = 0$ otherwise. Define the *modularity matrix* $B := A - \frac{\mathbf{d}\mathbf{d}^\top}{2m}$. Then modularity can be written as

$$Q(C) = \frac{1}{2m} \operatorname{Tr}(C^\top B C).$$

This spectral formulation serves as the foundation for many continuous and learning-based approaches.

Graph neural networks. Graph neural networks (GNNs) are a class of neural architectures that compute vector representations for vertices in an attributed graph by iteratively aggregating information from local neighborhoods. In the context of community detection, GNNs are used to map vertices into a representation space in which structurally or semantically related vertices are close. Learning-based clustering methods then infer communities indirectly by optimizing objectives such as continuous or differentiable relaxations of modularity, mutual-information maximization, or contrastive criteria defined over these representations or over derived *soft cluster assignments*, which encode the strength of association between nodes and communities.

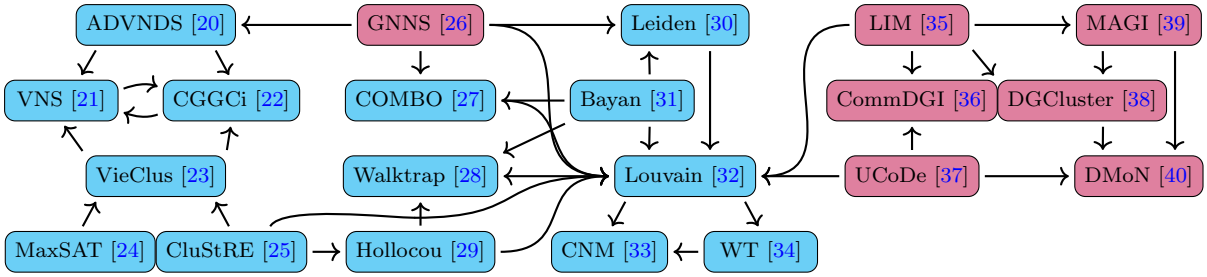


Figure 1: Previous experimental comparisons visualized. Each directed edge indicates that the source algorithm used the destination algorithm in the experimental evaluation. The colors indicate either a learning-based method in red, or a classical optimization algorithm in blue.

3 Background and Methods

We review objective functions and algorithmic paradigms for community detection and introduce the taxonomy used in this work.

Many clustering quality measures have been proposed, including *coverage*, *performance* [41], *inter-cluster conductance* [42], *surprise* [43], and *modularity* [8]. Coverage, the fraction of intra-cluster edges, is simple but favors trivial solutions. Modularity mitigates this by comparing observed coverage to its expectation under a null model, and has become one of the most widely used objectives for community detection [11]. It is the primary quality function in large-scale benchmarks such as the DIMACS challenges [10] and is the focus of this work. While Aref et al. [16] report that modularity heuristics rarely attain optimal solutions, Ansótegui et al. [24] show that the VieClus modularity maximization heuristic routinely computes optimal solutions on small to medium-sized graphs. Alternative objectives capture complementary notions of community structure: *Weighted Community Cluster* (WCC) maximizes the number of triangles within each community [44], implemented in the Scalable Community Detection algorithm [45], and the *Map Equation*, which measures the expected description length of a random walk [46], used in Infomap [46], Infoflow [47], and the recent learning-based Neuromap [12]. We do not include all the methods based on alternative objective functions in order to maintain our focus on modularity maximization.

Classical modularity-based algorithms use discrete optimization heuristics that consider graph topology only. More recently, learning-based approaches to community detection have been proposed, motivated by the development of GNNs that can model node attributes as feature vectors to capture additional graph information beyond topology alone. These methods typically optimize training objectives that are explicitly aligned with modularity or related structural principles. Compared to classical heuristics, learning-based methods trade exact modularity gains for flexibility, the ability to incorporate node attributes, and increased generalization across graphs. Despite extensive work in both paradigms, existing surveys do not provide a unified experimental comparison of classical and learning-based modularity optimization methods. Prior surveys typically focus on either classical heuristics [16] or learning-based methods [14, 15, 18, 19]. Li et al. [17] compare classical and deep learning methods, but limit comparison to Python-based solvers, thus lacking a comprehensive comparison of the state-of-the-art. In addition, algorithmic papers in both paradigms rarely evaluate against methods outside their own class (see Figure 1).

This work addresses this gap by providing a comprehensive experimental comparison of modularity-based community detection algorithms under a unified framework. Here, we provide

Table 1: Overview of surveyed community detection methods and software characteristics. The upper block lists the 17 methods integrated into our unified benchmarking framework; the lower block lists surveyed methods whose implementations are unavailable or not evaluated. *Code* reflects original software maturity prior to our integration ($\star\star$: production/package release with standard I/O; $\star$: research prototype adapted into our harness; $\times$: unreleased/unavailable). *Parallel* indicates multi-threading or GPU support; *Fixed k* denotes whether the number of clusters must be specified a priori; *Features* indicates utilization of node attributes; and *Py* indicates availability in standard Python package repositories. Bold denotes exact solvers.

Method	Year	Code	Source	Parallel	Fixed k	Features	Py
Benchmarked Methods (Integrated into Framework)							
CNM [33]	2004	$\star\star$	[48]	–	–	–	✓
Walktrap [28]	2005	$\star\star$	[49]	–	–	–	✓
Louvain [32]	2008	$\star\star$	[49]	–	–	–	✓
Infomap [46]	2008	$\star\star$	[49]	–	–	–	✓
COMBO [27]	2014	$\star\star$	[50]	–	–	–	✓
Hollocou [29]	2017	$\star\star$	[51]	–	–	–	–
VieClus [23]	2018	$\star\star$	[52]	✓	–	–	✓
Leiden [30]	2019	$\star\star$	[49]	–	–	–	✓
DMoN [40]	2020	$\star$	[53]	✓	✓	✓	–
CommDGI [36]	2020	$\star$	[54]	✓	✓	✓	–
GNNS [26]	2022	$\star$	[55]	✓	–	–	–
UCoDe [37]	2023	$\star$	[56]	✓	✓	✓	–
DGCluster [38]	2023	$\star$	[57]	✓	–	✓	–
MAGI [39]	2024	$\star$	[58]	✓	✓	✓	–
Bayan [31]	2024	$\star\star$	[59]	–	–	–	–
CluStRE [25]	2025	$\star\star$	[60]	–	–	–	–
LIM [35]	2025	$\star$	[61]	✓	–	✓	–
Surveyed Literature Baselines (Unavailable / Not Benchmarked)							
WT [34]	2007	$\times$		–	–	–	–
CGGCi-RG [22]	2012	$\times$		–	–	–	–
VNS [21]	2012	$\times$		–	–	–	–
ADVNDS [20]	2017	$\times$		–	–	–	–
MaxSAT [24]	2025	$\times$		–	–	–	–

a taxonomy of the considered methods listed in Table 1. We organize methods by *optimization paradigm*, distinguishing between *classical* combinatorial algorithms and *learning-based* (or GNN-inspired continuous) approaches. Classical methods are further grouped by their dominant optimization mechanism (e.g., greedy, multilevel, or metaheuristic) and by computational setting (e.g., streaming and exact solvers). Learning-based methods are categorized by how modularity is used during training, separating approaches that directly optimize continuous relaxations of modularity from those that use modularity as a guiding signal within broader learning objectives. Within each subgroup, algorithms are listed chronologically.

3.1 Classical Modularity Optimization Methods

3.1.1 Greedy agglomeration and multilevel local search

Early approaches to modularity maximization focused on greedy agglomeration, merging communities to directly increase modularity. Subsequent work identified limitations of purely agglomerative strategies, notably unbalanced merges and poor scalability, shifting toward multilevel formulations based on local vertex moves.

Clauset–Newman–Moore (CNM) [33] is a hierarchical agglomerative algorithm that greedily maximizes modularity. Starting from singleton communities (each vertex assigned to its own cluster), it repeatedly merges the pair of communities with the largest modularity gain. The algorithm produces a dendrogram representing the full merge hierarchy and returns the clustering attaining maximum modularity. Its running time is $O(md \log n)$, where d is the dendrogram depth.

Wakita–Tsurumi (WT) [34] refines CNM by addressing its tendency to produce highly unbalanced merges that degrade scalability on large graphs. While preserving the same agglomerative structure, WT modifies merge selection to account for both modularity gain and a community-size consolidation ratio, biasing the process toward more balanced agglomerations and improving scalability.

Louvain [32] replaces community-pair merging with greedy local vertex moves. Vertices are iteratively moved to neighboring communities whenever the induced modularity gain is positive. Once no improving move exists, communities are contracted into supernodes and the process is repeated, yielding a multilevel optimization scheme. Each local-moving phase runs in time linear in the number of edges, and the number of multilevel passes is typically small, resulting in near-linear runtime in practice.

Leiden [30] addresses LOUVAIN's tendency to produce disconnected communities by introducing a refinement phase before contraction. This phase splits and restructures communities through probabilistic, non-greedy merges, while guaranteeing that resulting communities are connected and locally optimal.

3.1.2 Non-modularity baselines

We include two widely used non-modularity baselines which rely on random walks to model how information or flow propagates through the network, providing an alternative perspective to modularity optimization.

Walktrap [28] performs hierarchical clustering based on the intuition that short random walks on a graph tend to get "trapped" into densely connected regions corresponding to communities. Each vertex is characterized by the probability distribution over vertices reached by a random walk starting from it, and a structural similarity distance between vertices (and communities) is defined by comparing these distributions. Communities are then obtained by agglomerative merging of the closest pairs under this distance, producing a dendrogram similar to CNM, which is then cut at the partition attaining maximal modularity. The algorithm runs in $O(mnd)$ time, where d is the depth of the resulting dendrogram.

Infomap [46] formulates community detection as an information-theoretic problem, seeking a clustering that minimizes the expected description length of a random walk under the map equation. It uses a LOUVAIN-inspired greedy multilevel search, augmented with fine- and

coarse-tuning steps (submodule and supermodule movements) to escape local minima.

3.1.3 Metaheuristics and ensemble/hybrid search

Beyond greedy and multilevel heuristics, several methods apply metaheuristic or ensemble-learning strategies to modularity maximization. Representative examples include Variable Neighborhood Search (VNS) [21] and its decomposition-based extension ADVNDS [20], which systematically perturb and decompose the current clustering to escape local optima, as well as ensemble-based approaches such as CGGCi-RG [22], which stabilize modularity optimization by consolidating recurring community structures across multiple runs. These methods combine diversification mechanisms (e.g., randomized perturbations or multiple initializations) with intensification via modularity-based local refinement. While more computationally expensive than purely greedy algorithms, such strategies often achieve higher modularity by exploring a larger search space.

COMBO [27] considers pairs of communities and evaluates full redistributions of vertices between them using Kernighan–Lin-style refinements, instead of relocating individual vertices as in Louvain. By unifying merges, splits, and partial recombinations, COMBO can escape local optima that trap multilevel greedy methods, at the cost of higher computational complexity. The authors report an upper bound on running time of $O(n^2 \log C)$, where C is the number of clusters.

VieClus [23] further extends ensemble-based optimization by adopting a memetic (evolutionary) framework. Rather than refining a single partition, it maintains a population of clusterings and generates new solutions through recombination and mutation. Recombination exploits agreement between high-quality solutions via overlay clustering, where two vertices are grouped together only if they co-occur in both parent solutions, while mutation introduces controlled perturbations to preserve diversity. Each offspring is refined using Louvain or multilevel local search and retained only if it improves modularity. After a set time limit, VieClus returns the best clustering found thus far.

3.1.4 Streaming algorithms

Most classical modularity algorithms assume random access to the full graph, which becomes infeasible for large or dynamically generated networks. Streaming algorithms address this limitation by processing the graph incrementally under strict memory constraints, typically allowing only a single or few passes over the data.

Hollocou [29] is a one-pass edge-streaming algorithm. The algorithm starts from the singleton clustering and processes edges sequentially. For an arriving edge (u, v), Hollocou decides whether one endpoint should join the other's community based on their current community volumes. If both volumes are below a fixed threshold, the node belonging to the smaller-volume community joins the other. The authors show that under random stream orderings, this local, volume-based assignment rule corresponds to a greedy optimization of modularity in the streaming setting.

CluStRE [25] follows the node-streaming model unlike the edge-streaming model of Hollocou. Here, nodes arrive sequentially together with their neighborhood lists. Starting from singleton clustering, each node is assigned to the cluster among its neighbors that maximizes the modularity gain similar to a Louvain local move. In its multi-stage configurations, a quotient

graph is constructed during streaming whose nodes represent clusters and whose weighted edges summarize inter- and intra-cluster connectivity, preserving modularity exactly; this quotient graph is optionally refined using VieClus, and the graph may be re-streamed until convergence. In our benchmark, we evaluate the one-pass *light* streaming configuration to measure pure streaming performance.

3.1.5 Exact solvers

In contrast to heuristic approaches, exact algorithms provably compute globally optimal modularity clusters. Since modularity maximization is NP-hard [62], these methods rely on exponential-time search augmented with bounding techniques or logical formulations such as weighted partial MaxSAT [24], and are therefore limited to small or medium-sized graphs.

Bayan [31] is an exact branch-and-cut algorithm for modularity maximization. It formulates modularity as an integer program with binary co-membership variables and enforces consistency via triangular constraints on node triples, which branch on whether three nodes must belong to the same community or cannot all be clustered together. Upper bounds are obtained by solving linear programming relaxations, while lower bounds are computed from feasible clusterings produced by COMBO. Branching decisions either contract node triples into supernodes or discourage their co-assignment, and the algorithm terminates once optimality is certified or a prescribed optimality gap is reached.

3.2 Learning-Based Methods

3.2.1 Continuous and differentiable modularity maximization

These approaches replace discrete community assignments with continuous representations, often learned using GNNs, expressing modularity as a continuous or differentiable objective optimized over node representations or soft cluster assignments.

DMoN [40] introduces a differentiable relaxation of modularity and realizes it using a GNN. A GNN followed by a softmax function produces soft cluster assignments, which are optimized using an objective whose main term corresponds to modularity in spectral form. The objective further includes a *collapse regularization* term that prevents trivial solutions in which all nodes are assigned to the same cluster.

GNNS [26] is a GNN-inspired algorithm that formulates modularity maximization as a recurrent continuous optimization process directly over soft cluster assignments (without trainable neural weights or node features). Starting from random assignments, node assignments are iteratively updated by combining previous assignments with modularity-weighted aggregation of neighboring assignments. Activation functions and normalization steps amplify valid assignments supported by positive modularity contributions. The procedure is repeated for multiple random initializations and parameter choices, and the best converged solution is discretized to obtain the final clustering.

Most learning-based modularity methods, including DMoN, require the number of communities to be specified a priori. **DGCluster** [38] eliminates this requirement by learning node embeddings with a GNN and clustering implicitly through a joint optimization objective. Node embeddings learned by a GNN are projected onto the positive unit hypersphere to induce soft pairwise community memberships, which parameterize a differentiable modularity objective.

A secondary loss optionally incorporates auxiliary local information, such as partial labels or clusterings from classical heuristics, by aligning embedding similarities with a pairwise similarity matrix. After end-to-end optimization, final communities are obtained by clustering the learned embeddings using BIRCH [63].

While DGCluster removes the need to predefine the number of communities, it relies on jointly optimizing multiple objectives which increases model and training complexity. **LIM** [35] addresses this trade-off with a single-objective formulation that still does not require prior specification of the number of communities. The method first applies Louvain to extract coarse global communities, which are summarized as structural community centers in a feature space. A graph convolutional network then learns node representations that capture local structural information and node attributes. Soft cluster assignments are computed by measuring similarity between node representations and the structural centers, and a single modularity objective defined over these assignments is optimized to obtain the final clustering.

3.2.2 Contrastive/infomax objectives

These methods do not optimize modularity directly, but instead use it to guide representation learning through alternative objectives.

Community Deep Graph Infomax (CommDGI) [36] learns community-aware node representations by jointly maximizing mutual information between nodes and the graph and between nodes and their communities. Unlike other methods that directly optimize modularity, modularity is used here as an auxiliary objective together with mutual information, via a soft k-means-style clustering layer, to encourage structurally coherent communities.

Unified Community Detection (UCoDe) [37] optimizes a single contrastive objective over the community-wise modularity matrix $C^\top BC$, encouraging high intra-community modularity along the diagonal and low inter-community values off the diagonal. This contrastive formulation allows UCoDe to recover non-overlapping communities while naturally supporting overlap, without requiring separate objectives or post-processing. Compared to combined-objective methods such as CommDGI, UCoDe uses a single, interpretable loss and requires minimal hyperparameter tuning.

MAGI [39] establishes a formal connection between modularity maximization and graph contrastive learning by showing that modularity naturally defines positive and negative node pairs via the sign of the modularity matrix. Specifically, node pairs with positive modularity coefficients are treated as positive samples, while pairs with non-positive coefficients are treated as negatives, yielding a contrastive objective equivalent to relaxed modularity maximization. A GNN encoder is trained using a community-aware contrastive loss, with mini-batch modularity matrices constructed via random-walk sampling for scalability. In our benchmark, we evaluate the authors' scalable GraphSAGE minibatch implementation; their alternative full-graph GCN formulation constructs a dense $n \times n$ affinity matrix, incurring an $\mathcal{O}(n^2)$ computational footprint that is prohibitive on larger graphs.

4 Evaluation Framework

In this section, we describe the benchmarking methodology used to evaluate modularity-based graph clustering algorithms under controlled and reproducible conditions. A central contribution

is the integration of 17 clustering algorithms behind a unified benchmarking interface, offering a fair comparison across diverse implementations. In addition to enabling the experiments presented in this work, it will also serve as an openly available resource for future work on graph clustering, simplifying the evaluation of older methods on new data or machines. This section also includes details on the datasets we used to evaluate the clustering algorithms.

4.1 Benchmarking Interface

As shown in Table 1, over half of the clustering algorithms in the literature lack ready-to-use interfaces for clustering new data. Furthermore, among those with properly documented code, there are wide differences in the input graph format, output formats, and the additional information they report, such as time and memory consumption. This lack of standardization complicates fair benchmarking, as differences in preprocessing, reporting, and resource accounting can substantially affect observed performance.

To address this problem, we aim to provide an interface with a single common input and output format and consistent time and memory monitoring. The interface exposes a common set of inputs, while allowing solver-specific parameters to be provided when applicable:

$$\textsc{Cluster}(G, T_{\max}, p, k, M_{\max}, \mathcal{F})$$

where G is the input graph, $T_{\max}$ is the wall-clock time limit, p is the number of threads available to the algorithm, k is the number of clusters to compute, $M_{\max}$ is the memory limit in gigabytes, and $\mathcal{F}$ are the graph features. Parameters p, k, and $\mathcal{F}$ are only used by a subset of the solvers. From Table 1, parallel solvers use p, those that compute a fixed number of clusters use k, and any solver that uses graph features can optionally accept $\mathcal{F}$. When $\mathcal{F}$ is not provided by the dataset, but the solver expects features, we generate a 69-dimensional synthetic feature vector (`ldp+rni`) for each vertex by concatenating a 5-dimensional standardized Local Degree Profile (vertex degree and the minimum, maximum, mean, and standard deviation of neighbor degrees) with a 64-dimensional seeded Gaussian Random Node Initialization, providing structural and node-distinguishing signals without quadratic memory overhead. Similarly, when a solver requires a target cluster count k, labeled Ground-Truth graphs provide $|C|$ (or $\max(16, |C|)$ for over-provisioning methods DMoN, UCoDe, and GNNS, following their original evaluation protocols), while unlabeled DIMACS graphs use $k = \lceil \sqrt{n} \rceil$. This interface is designed to capture the most common operational requirements across clustering algorithms while enforcing consistent resource limits and output reporting.

Whenever possible, we use the authors' implementations or the most commonly used library implementation. The exact setup for each algorithm is described in our openly available GitHub repository[1]. We make only minimal, deliberate modifications where necessary to ensure consistent timing, memory accounting, and output quality measures based on the following rules:

- Parsing the input and writing the output to a file is not counted in the timing of the algorithm.
- For search-based algorithms that always run for the entire $T_{\max}$ duration, the recorded computation time is defined as the point in time when the best solution is found.

[1] https://github.com/ClusterBenchmark/ClusterBenchmark

Table 2: Default architectures and training hyperparameters for the benchmarked neural clustering methods under Protocol A.

Method	Hidden Layers	Epochs	LR	Method-Specific Hyperparameters
DMoN	$1 \times (512)$	1 000	0.001	collapse regularization = 1.0, dropout = 0.5
CommDGI	$1 \times (64)$	1 000	0.001	$\alpha = 2.0$, $\beta = 5.0$, temperature = 30
UCoDe	$1 \times (256)$	1 000	0.001	weight decay = 0.1
DGCluster	$2 \times (256, 128)$	300	0.001	GCN backbone, BIRCH threshold = 0.5
MAGI	$2 \times (256, 256)$	400	0.010	samples $(10, 10)$, walks $= 20 \times 4$, $\tau = 0.5$
LIM	$1 \times (512)$	300	0.001	Louvain res. = 0.3, temp. = 30, $\alpha = 0.001$

- Learning-based methods that train a neural network as part of the algorithm will break the training loop at $T_{\max}$, and use the current model at that time to produce the final clustering.

4.2 Parameter Settings

Where possible, we use the default configurations for all algorithms. To reflect how algorithms are typically used in practice, our primary evaluation (**Protocol A**) deliberately avoids any dataset-specific or instance-specific parameter tuning, benchmarking available implementations out-of-the-box on a single CPU thread with their author-recommended defaults. To isolate the effects of hardware resources and hyperparameter sensitivity, we complement Protocol A with two targeted protocols: **Protocol S** (scalability and accelerators), which evaluates full-machine multi-threaded CPU execution (32 threads) and GPU acceleration while holding algorithm hyperparameters fixed at their defaults; and **Protocol B** (per-instance parameter tuning), which runs a 50-trial Bayesian hyperparameter search (via Optuna) maximizing modularity on representative small graphs.

Default configurations (Protocol A). Among classical and topology-only methods (CNM exposes no tunable parameters), Louvain, COMBO, Leiden ($\beta = 0.01$, 2 iterations), and Bayan (using the Gurobi LP/ILP solver [64] with optimality gap 10^{-3}) all use a default resolution of 1. Walktrap uses a random-walk length of 4, Infomap uses 1 outer trial, Hollocou sets the maximum volume threshold to $v_{\max} = 10\,000$, CluStRE runs in its one-pass light streaming mode optimizing modularity, VieClus runs with a 1-hour time limit, and GNNS uses the GNNS100 configuration (100 random initializations, keep fraction 1/3). The default architectures and training hyperparameters for the six neural methods are summarized in Table 2.

For stochastic algorithms with an explicitly specified seed, we use the run ID as the seed for each of the five runs. While many of these algorithms include multiple configurations, we use either the configuration recommended by the authors or the default configuration provided with the implementation.

4.3 Dataset

We consider three datasets commonly used in the literature. The first set consists of 10 real-world graphs with known community labels; we refer to this dataset as the Ground-Truth graphs. The machine learning algorithms commonly use these graphs, and the details of the instances are shown in Table 3. Next, we consider the instances used in the 10th DIMACS implementation

Table 3: GROUND-TRUTH graphs. The $|C|$ column shows the number of clusters and x the number of features.

Instance	n	m	x	$\|C\|$
Cora	2 708	5 278	1 433	7
Citeseer	3 327	4 552	3 703	6
Amazon-Photo	7 650	119 081	745	8
Amazon-Computers	13 752	245 861	767	10
Coauthor-CS	18 333	81 894	6 805	15
Pubmed	19 717	44 324	500	3
Coauthor-Physics	34 493	247 962	8 415	5
ogbn-Arxiv	169 343	1 157 799	128	40
Reddit	232 965	57 307 946	602	41
ogbn-Products	2 449 029	61 859 012	100	47

challenge on graph clustering [10]; we call this dataset DIMACS graphs. This dataset is popular among the classical modularity optimization methods, and the details of these graphs are shown in Appendix A. Finally, we use synthetically generated graphs with node features generated using the attributed, degree-corrected stochastic block model (ADC-SBM) [65]. This generator plants k clusters in the graph, and generates random edges via a distribution that is conditional on those clusters. It does so using the two input parameters d and $d_{out} \leq d$, where d is the expected degree for each vertex and d_{out} is the expected number of neighbors for a node in a different cluster than its own. The ADC-SBM generator also produces node features that are correlated with the community structure. It does so using the two input parameters σ for intra-cluster variance and σ_c for cluster center variance.

5 Experiments

In this section, we present benchmark results obtained using the unified benchmarking interface described in Section 4. We report results on real-world graphs with and without ground-truth labels, as well as on synthetically generated graphs with controlled structural and feature noise. Rather than exhaustively ranking algorithms, these experiments illustrate how different algorithmic design choices behave under a standardized benchmarking setup. To support this, we also provide a dynamic website reporting updated results[2].

5.1 Computational Environment

All algorithms are evaluated on identical hardware nodes to eliminate variability and ensure fair comparisons. Each machine is equipped with a 16-core (32-thread) Intel Xeon w5-3435X processor, 125 GiB of system memory, and an NVIDIA GeForce RTX 4090 GPU with 24 GiB of VRAM (driver version 560.35.03, CUDA 12.6), running Ubuntu 24.04.3 LTS with Linux kernel 6.8.0-107. Compiled C/C++ implementations are built with GCC 13.3.0, and Python implementations run on Python 3.12.13.

To prevent resource contention and cross-talk, each benchmark job runs exclusively on a node under Linux `cgroup` memory tracking. Under our primary out-of-the-box evaluation

[2] https://clusterbenchmark.github.io/

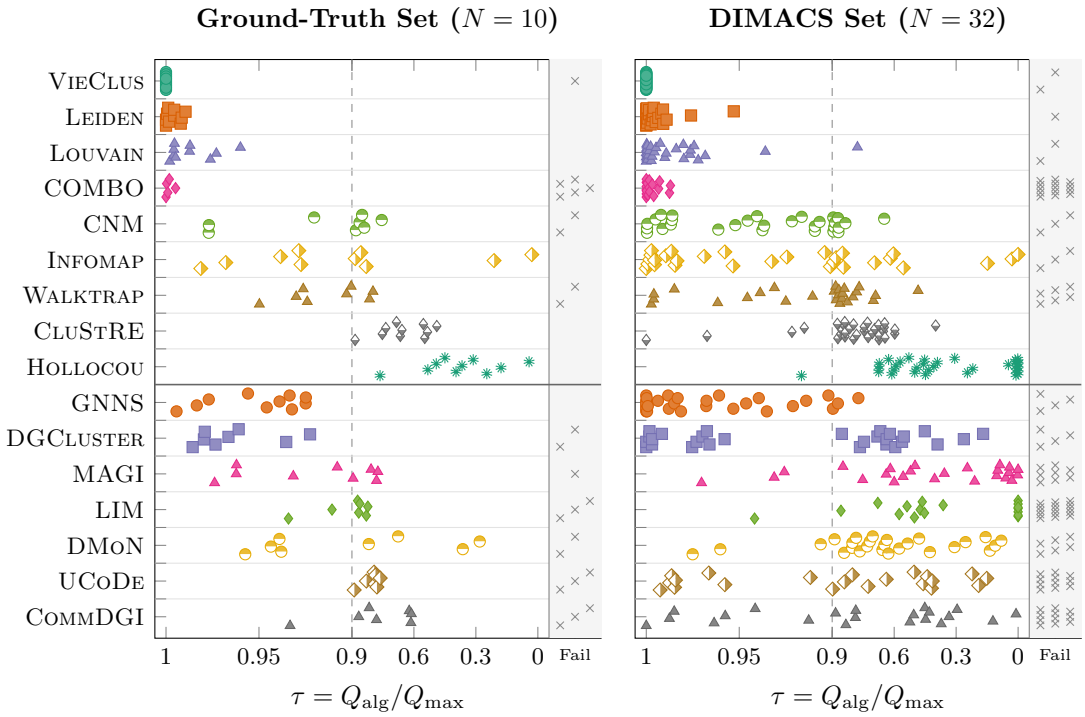


Figure 2: Modularity (Q) performance distributions under Protocol A (out-of-the-box defaults, single-threaded CPU) on the Ground-Truth ($N = 10$, left) and DIMACS ($N = 32$, right) datasets. Each horizontal lane corresponds to one algorithm (classical heuristics above the solid horizontal line; continuous and learning-based methods below). Colored markers show solved instances at their relative modularity ratio $\tau = Q_{\text{alg}}/Q_{\max}$ (with vertical jitter to separate overlapping points), while gray $\times$ markers in the right-hand *Fail* band indicate instances that exceeded the time or memory limit (or collapsed to a trivial single cluster). The vertical dashed line at $\tau = 0.90$ separates the high-precision regime ($\tau \in [0.90, 1.0]$) from the tail ($\tau \in [0.0, 0.90]$).

(**Protocol A**), all 17 algorithms are restricted to a single CPU thread ($p = 1$, `device=cpu`) to establish a uniform sequential baseline across both single-threaded and parallel implementations. Under **Protocol S**, we evaluate the parallel and neural methods using all 32 hardware threads on CPU ($p = 32$, `device=cpu`) as well as GPU acceleration ($p = 32$, `device=cuda`) for CUDA-enabled architectures. Each algorithm–instance run is subject to a wall-clock time limit of one hour (3 600 s) and a host memory limit of 120 GB (or the 24 GiB VRAM ceiling when executing on the GPU). Runs exceeding the memory limit are terminated and recorded as *memory limit exceeded* (MLE). Time limits include a 50% watchdog grace period (4 500 s) so that fixed-epoch neural training loops can terminate cleanly; runs exceeding the one-hour budget are recorded as *time limit exceeded* (TLE).

For stochastic methods, we execute five independent runs seeded by the run index and report the best solution quality achieved. In addition to modularity (Section 2), we evaluate ground-truth community recovery using Normalized Mutual Information (NMI) [66] and an edge-restricted pairwise F1 score. The F1 score is computed over adjacent vertex pairs: an edge (u, v) is a true positive (TP) if u and v are assigned to the same cluster by the algorithm as well as in the ground-truth partition. Defining false positives (FP) and false negatives (FN) analogously over graph edges, $F1 := \frac{2\,TP}{2\,TP+FP+FN}$.

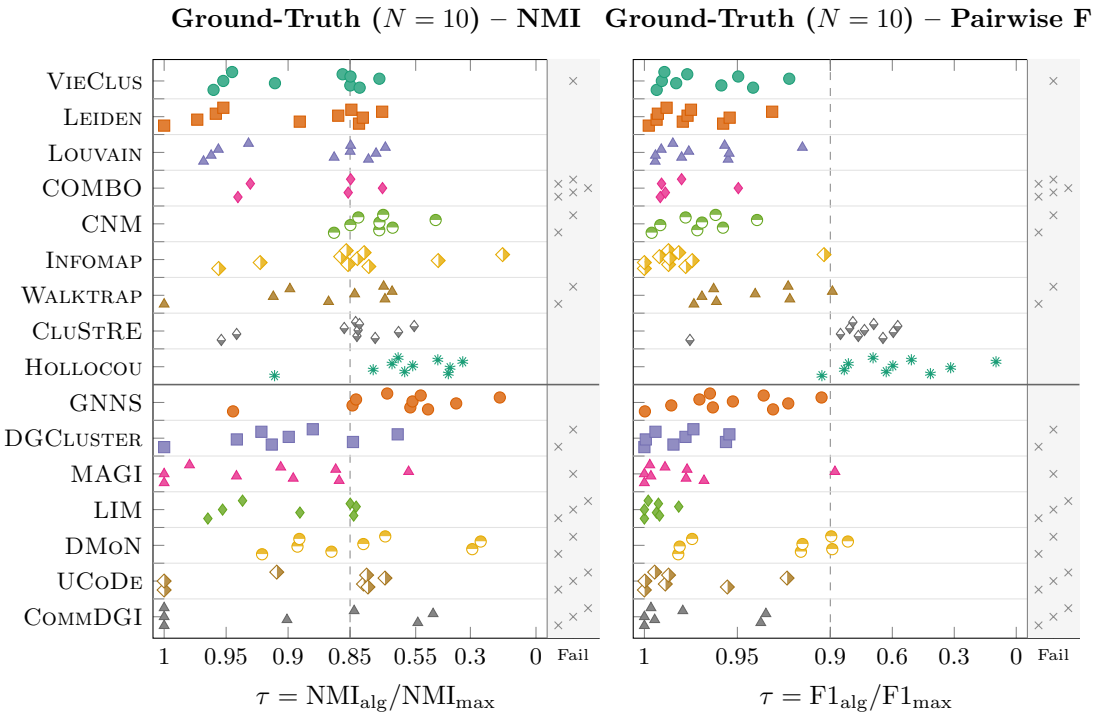


Figure 3: Ground-truth community recovery distributions under Protocol A on the Ground-Truth dataset ($N = 10$), measured by Normalized Mutual Information (NMI, left) and edge-restricted Pairwise F1 (right). Colored markers denote solved instances at their relative recovery ratio τ, and gray $\times$ markers in the right-hand *Fail* band denote unresolved instances.

Relative performance distributions. To compare multiple algorithms across heterogeneous benchmark instances without sensitive raw-score averaging, we follow the relative-ratio normalization of Dolan and Moré [67]. For each benchmark instance and quality metric, an algorithm's score is normalized by the best score observed across all evaluated methods on that instance, yielding a relative quality ratio $\tau \in [0, 1]$. Rather than superimposing 16 cumulative step functions that cross when algorithms time out, we display a horizontal strip lane for each algorithm in which every solved instance is plotted as an individual marker at its ratio τ (with small vertical jitter to separate overlapping points near $\tau = 1$), and unresolved instances (TLE/MLE) are shown as $\times$ markers in a dedicated *Fail* band on the right. Algorithms whose markers cluster tightly near $\tau = 1$ with few or no entries in the *Fail* band achieve both high solution quality and broad instance coverage.

5.2 State-of-the-Art Comparisons (Protocol A)

Figures 2 and 3 present the out-of-the-box Protocol A performance profiles for modularity optimization and ground-truth community recovery, respectively. Across both the attributed Ground-Truth graphs and the topology-only DIMACS suite (Figure 2), classical modularity heuristics clearly dominate pure modularity maximization. VieClus achieves the highest modularity on every single instance it completes ($\tau = 1.0$ on 9/10 Ground-Truth graphs and 30/32 DIMACS graphs), consistent with prior findings [23, 24], failing only on the largest instances (`ogbn-products`, `uk-2002`, and `uk-2007-05`) due to time or memory limits. Leiden and Louvain follow closely behind: Leiden solves all 10 Ground-Truth instances with

$\tau \geq 0.990$ and 31/32 DIMACS instances with $\tau \geq 0.953$. Among continuous and learning-based methods, only DGCluster ($\tau \geq 0.923$ on 8/10 Ground-Truth graphs) and the topology-driven GNNS ($\tau \geq 0.925$ on 10/10 Ground-Truth graphs and 28/32 DIMACS graphs) approach the modularity of classical heuristics, whereas contrastive and deep GNN models lag substantially in raw modularity, particularly on the featureless DIMACS graphs where they rely on synthetic degree-profile and random-node-initialization (`ldp+rni`) features.

When evaluating ground-truth community recovery on the Ground-Truth dataset (Figure 3), the relative standing shifts markedly. Here, attribute-aware learning-based methods are highly competitive with—and frequently surpass—pure modularity maximizers. In terms of NMI (Figure 3, top), MAGI achieves the strongest overall profile (mean NMI of 0.533 across 9/10 solved instances, with $\tau \geq 0.859$ on 80% of instances), while CommDGI, DGCluster, and UCoDe each achieve the best NMI ($\tau = 1.0$) on multiple individual networks alongside Leiden. On edge-restricted Pairwise F1 (Figure 3, bottom), MAGI, LIM, CommDGI, DGCluster, UCoDe, and the flow-based Infomap all attain $\tau = 1.0$ on top instances. This divergence between Figure 2 and Figure 3 underscores a fundamental misalignment between topological modularity and externally defined node labels: for example, on Citeseer, the ground-truth label partition has a modularity of only 0.54, whereas classical and neural modularity maximizers find partitions with modularity up to 0.89. Incorporating node attributes allows GNNs to align communities with semantic categories even when citation edges cross category boundaries.

The streaming algorithms Hollocou and CluStRE trade solution quality for extreme scalability, solving all 10 Ground-Truth and all 32 DIMACS graphs—including `uk-2007-05` (3.3 billion edges)—with CluStRE consistently outperforming Hollocou in both modularity and NMI/F1. On the eight smallest DIMACS instances where exact optimal modularity partitions are known [24, 31], VieClus recovers the optimal modularity on all 8/8 instances, Leiden and GNNS on 6/8, COMBO on 5/8, and Louvain and DGCluster on 2/8.

5.3 Quality–Efficiency Trade-offs and Hardware Scaling (Protocol S)

Figure 4 summarizes the trade-off between solution quality and execution time under Protocol A using geometric means across solved instances, with per-graph runtimes normalized by the number of edges ($\mu s/m$). Across both datasets, the algorithms span six orders of magnitude in normalized runtime. Streaming methods (Hollocou at 0.04–0.05 $\mu s/m$ and CluStRE at 0.26–0.64 $\mu s/m$) occupy the extreme low-latency regime, whereas multi-level local-moving heuristics (Leiden at 0.78–0.85 $\mu s/m$ and Louvain at 1.74–2.12 $\mu s/m$) dominate the Pareto frontier, achieving near-optimal modularity more than four orders of magnitude faster than the evolutionary solver VieClus (which intentionally exhausts its configurable time budget to refine partitions). Among continuous and learning-based methods, the sparse matrix-free GNNS (375 $\mu s/m$ on Ground-Truth) is an order of magnitude faster than deep neural architectures (4 300–26 400 $\mu s/m$), which incur substantial overhead from multi-epoch gradient optimization.

Peak memory consumption exhibits an equally sharp divide between algorithmic paradigms (detailed per-instance measurements are provided in Appendix B, Table 6). Streaming methods process edges in a single pass with minimal state, requiring only 1.1 GB on `ogbn-products` (61.9 M edges), while Leiden and Louvain require 12.7 GB. In contrast, learning-based GNNs store high-dimensional node embeddings and intermediate activation tensors, consuming 14–47 GB on medium-to-large graphs. Furthermore, implementations that materialize dense

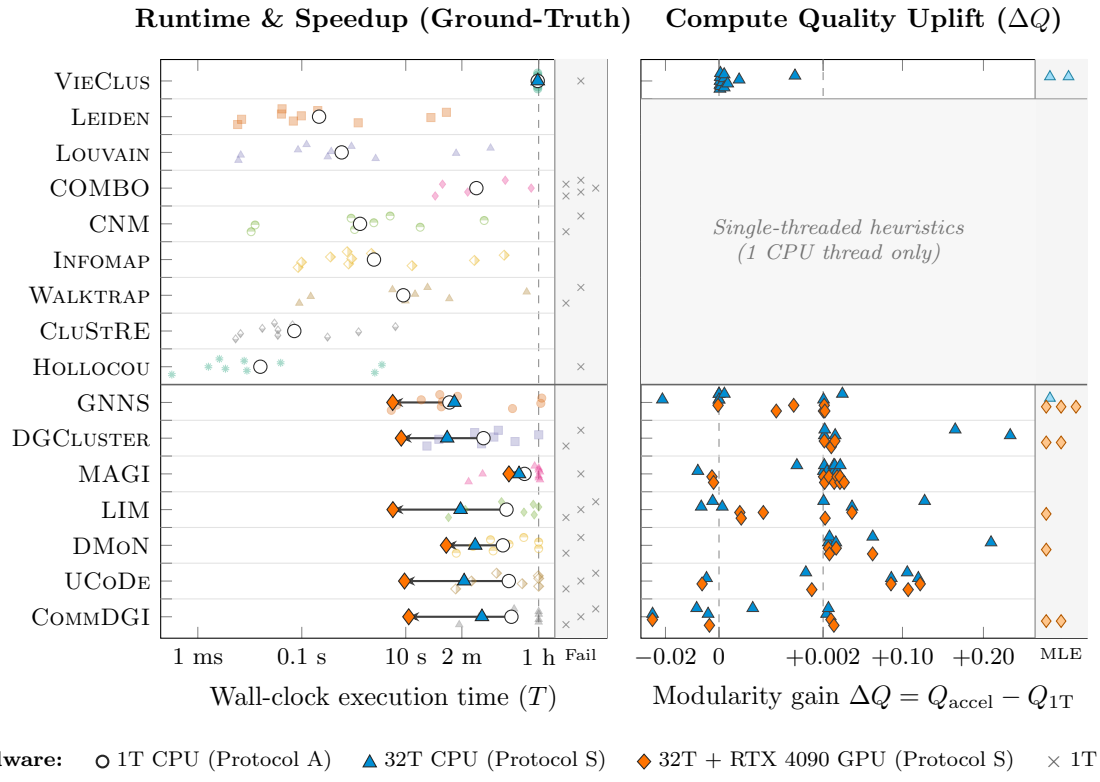


Figure 4: Runtime efficiency and hardware scaling across all 16 non-exact algorithms (classical heuristics above the solid line; continuous and learning-based methods below). **Left:** Wall-clock execution time distributions on the Ground-Truth dataset ($N = 10$). Faint markers show individual solved instances under 1T CPU (Protocol A), while bold arrows trace geometric-mean runtime reductions from 1T CPU (○) to 32T CPU (▲) and RTX 4090 GPU (◆) across commonly solved instances. **Right:** Modularity uplift $\Delta Q = Q_{\text{accel}} - Q_{1\text{T}}$ on compute-bound instances (where 1T CPU truncated training epochs at the 1-hour limit, or for VieClus anytime evolutionary refinement). Vertical dashed lines separate $\Delta Q = 0$, the fine-tuning regime ($\Delta Q \in [0, +0.002]$), and the epoch-rescue regime ($\Delta Q \in [+0.002, +0.25]$). Markers in the right-hand *MLE* band denote instances that solved on 1T CPU (120 GB RAM) but exhausted memory under 32-process MPI (VieClus) or the 24 GiB GPU VRAM ceiling.

$O(n^2)$ modularity or similarity matrices in host memory—specifically LIM, UCoDe, and CommDGI—attempt to allocate 214 GiB on `ogbn-arxiv` ($n = 169\,343$) and 43.6 TiB on `ogbn-products`, resulting in immediate memory exhaustion (MLE).

To quantify how multi-core parallelism and hardware accelerators shift this efficiency frontier, **Protocol S** evaluates the parallel and learning-based solvers using all 32 hardware threads on CPU ($p = 32$, `device=cpu`) and on the NVIDIA RTX 4090 GPU ($p = 32$, `device=cuda`); complete per-instance runtimes are reported in Appendix C (Table 8). Expanding from 1 to 32 CPU threads yields geometric-mean speedups of 8.0× for UCoDe, 7.6× for LIM, 5.0× for DGCluster, 3.7× for CommDGI, and 3.4× for DMoN across medium-scale networks. Offloading training to the RTX 4090 GPU produces dramatic additional accelerations over single-threaded CPU execution, reaching geometric-mean speedups of 158× for LIM (e.g., reducing `coauthor-physics` from 3 615 s to 40.5 s), 100× for UCoDe, 95× for CommDGI, 37× for DGCluster, 13× for GNNS, and 12× for DMoN. Crucially, inspection of completed training epochs (Appendix B, Table 7) shows that while most neural methods already finish all

requested epochs within one hour on smaller graphs under single-threaded CPU execution, larger networks such as `coauthor-physics` and `ogbn-arxiv` prematurely truncate training under $p = 1$ (e.g., DGCluster completes 270/300 epochs on `ogbn-arxiv`, and LIM completes 111/300 on `coauthor-physics`) but finish 100% of their training epochs in under two minutes on the GPU. However, GPU acceleration introduces a stricter memory bottleneck: because the RTX 4090 is limited to 24 GiB of VRAM compared to 120 GB of system RAM, full-graph GNNs hit out-of-memory limits on smaller graphs on the GPU than on the CPU (for instance, CommDGI requires 29.7 GB on `coauthor-physics` and therefore succeeds on CPU but triggers MLE on GPU).

5.4 Hyperparameter Sensitivity (Protocol B)

Table 4: Modularity (Q) achieved under out-of-the-box defaults versus 50-trial Optuna Bayesian hyperparameter tuning (**Protocol B**) on Cora, Citeseer, and Pubmed. For fixed-k neural models tuned under the cluster-free regime ($k = \lceil\sqrt{n}\rceil$), the default baseline under the same cluster budget is reported alongside the tuning gain (ΔQ).

		Cora		**Citeseer**		**Pubmed**	
Method	Tuned Parameters	Def.	ΔQ	Def.	ΔQ	Def.	ΔQ
Leiden	γ	0.822	+0.001	0.893	−0.000	0.784	−0.000
Louvain	γ, tol.	0.815	−0.005	0.890	−0.001	0.771	+0.001
COMBO	γ, max com.	0.824	+0.000	0.893	−0.000	TLE	+0.740
Walktrap	walk steps t	0.765	+0.024	0.826	+0.028	0.711	+0.016
CluStRE	mode, passes, α, β, τ	0.550	+0.268	0.612	+0.277	0.386	+0.393
Hollocou	δ, $\text{vol}_{\max}$	0.406	+0.082	0.683	−0.010	0.355	−0.011
GNNS	γ, rank r, steps	0.780	+0.019	0.855	+0.015	0.736	+0.028
DGCluster	λ, lr, base model	0.815	−0.001	0.875	−0.001	0.738	+0.006
MAGI (SAGE)	τ, lr, wd, scale, arch.	0.786	+0.003	0.878	+0.001	0.641	+0.049
MAGI (GCN)	τ, lr, wd, scale, arch.	0.710	+0.064	0.817	+0.049	0.399	+0.106
LIM	$\gamma, \alpha, \beta, \nu$, lr	0.768	+0.002	0.800	+0.018	0.654	+0.011
CommDGI	T, α, β, lr, wd, hid.	0.758	+0.013	0.813	+0.009	0.003	+0.687

To assess how strongly algorithm rankings depend on hyperparameter tuning versus out-of-the-box defaults, **Protocol B** conducts 50-trial Bayesian optimization using Optuna [68] on the three canonical citation networks (Cora, Citeseer, and Pubmed), maximizing modularity Q. Table 4 compares default modularity scores alongside tuning gains (ΔQ) across methods with tunable algorithmic or loss parameters.

The results reveal a sharp dichotomy in hyperparameter robustness. Classical modularity-maximizing heuristics (Leiden, Louvain, and COMBO) exhibit virtually zero sensitivity ($|\Delta Q| \leq 0.005$), because their default resolution parameter ($\gamma = 1.0$) already aligns directly with standard Newman–Girvan modularity; on Pubmed, tuning COMBO's resolution and community cap also enables it to finish well within the one-hour limit ($Q = 0.740$). For Walktrap, tuning the random-walk length $t \in [2, 12]$ yields consistent moderate gains (+0.016 to +0.028). Enabling multi-pass refinement in CluStRE (`clustre-strong`) lifts its modularity by +0.268 to +0.393 over single-pass streaming (`clustre-fast`), bringing it within 0.005 of Leiden on all three citation networks.

Among learning-based methods, architectures whose loss functions balance multiple competing terms—or that are sensitive to learning rate and temperature scaling—benefit substantially from tuning. GNNS gains +0.015 to +0.028 by tuning its subspace rank and step count, while full-graph MAGI (GCN) improves by +0.049 to +0.106 and mini-batch MAGI (GraphSAGE) improves by +0.049 on Pubmed. Most strikingly, CommDGI—which optimizes a joint combination of deep graph infomax, k-means clustering temperature, and modularity—suffers from mode collapse under default parameters on Pubmed ($Q = 0.003$ at $k = \lceil\sqrt{n}\rceil = 141$, and $Q = 0.484$ at exact $k = 3$), but recovers $Q = 0.690$ after tuning its loss weights and cluster temperature. Notably, however, even after 50 trials of per-instance Bayesian optimization, no learning-based algorithm surpasses the untuned out-of-the-box modularity of VieClus or Leiden on any of the three networks.

5.5 Synthetic Graphs (ADC-SBM)

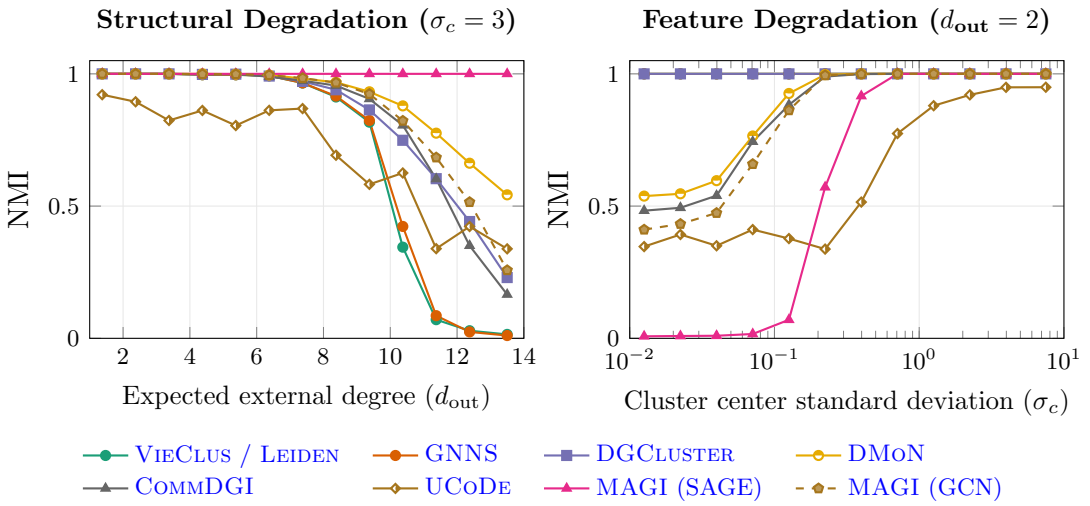


Figure 5: Ground-truth community recovery (NMI) on synthetic attributed networks generated with the ADC-SBM benchmark ($n = 1\,000$ vertices, $k = 4$ clusters, average degree $d = 20$, intra-cluster feature standard deviation $\sigma = 1$). Left: structural degradation via increasing expected external degree $d_{\text{out}} \in [1, 14]$ at fixed informative features ($\sigma_c = 3$). Right: feature degradation via decreasing cluster center separation $\sigma_c \in [10^{-2}, 10^{1}]$ at fixed strong community topology ($d_{\text{out}} = 2$). To reduce single-graph realization variance across the fine parameter grids ($\Delta d_{\text{out}} = 0.25$, 53 instances; 20 logarithmic steps per decade in σ_c, 61 instances), markers show means within non-overlapping bins of width $\Delta d_{\text{out}} = 1.0$ (4 graph instances per bin, left) and 0.25 dex (5 graph instances per bin, right). Leiden matches VieClus almost identically across both sweeps.

To isolate how clustering algorithms trade off topological structure against vertex attributes, we evaluate them on synthetic attributed networks generated using our ADC-SBM generator (Section 4). Rather than ranking solvers, these experiments reveal how different architectures respond to controlled degradation of structural and attribute signals. We conduct two high-resolution parameter sweeps on graphs with $n = 1\,000$ vertices, $k = 4$ clusters, average degree $d = 20$, and intra-cluster feature spread $\sigma = 1$: (i) a *structural degradation* sweep that increases the expected external degree d_{out} from 1 to 14 in steps of 0.25 (53 instances) while holding cluster features well-separated ($\sigma_c = 3$), and (ii) a *feature degradation* sweep that varies the cluster

center standard deviation σ_c logarithmically from 10^{-2} (uninformative overlapping features) to 10^{1} (well-separated features) at 20 steps per decade (61 instances) while holding the graph topology strongly modular ($d_{\text{out}} = 2$). Because each grid point represents an independent random graph realization, Figure 5 aggregates adjacent parameter steps into non-overlapping bins of width $\Delta d_{\text{out}} = 1.0$ (4 instances per marker) and 0.25 dex in σ_c (5 instances per marker).

Figure 5 summarizes both experiments. Under structural degradation (Figure 5, left), topology-only methods (VieClus, Leiden, and GNNS) maintain near-perfect recovery up to $d_{\text{out}} \approx 8$, after which they undergo a sharp phase transition as the stochastic block model approaches its detectability limit, dropping below NMI = 0.11 by $d_{\text{out}} = 11$. In contrast, attribute-aware GNNs leverage the informative node features ($\sigma_c = 3$) to sustain recovery well into the structurally noisy regime: at $d_{\text{out}} = 11$, DMoN achieves NMI = 0.849, MAGI (GCN) achieves 0.767, CommDGI achieves 0.724, and DGCluster achieves 0.720.

Comparing the two variants of MAGI—our default two-stage GraphSAGE encoder (MAGI-SAGE) versus the single-stage full-graph GCN encoder (MAGI-GCN)—exposes a striking architectural trade-off in message passing. Under structural degradation (Figure 5, left), MAGI-SAGE maintains NMI = 1.000 across the entire sweep up to $d_{\text{out}} = 14$, because GraphSAGE explicitly concatenates each vertex's ego-features with its sampled neighborhood representation, preserving clean ego-attribute separability even when 70% of incident edges cross cluster boundaries. Conversely, under feature degradation (Figure 5, right), when attributes become pure noise ($\sigma_c \leq 0.1$) on a clean graph topology ($d_{\text{out}} = 2$), MAGI-SAGE collapses to NMI ≈ 0.01–0.02, whereas MAGI-GCN retains NMI = 0.489–0.789 because full-graph degree-normalized Laplacian averaging smooths out zero-mean feature noise across well-connected communities. A similar spectrum appears across the other GNNs: DGCluster relies primarily on its pairwise modularity-similarity matrix and retains NMI = 1.000 across all $\sigma_c \in [10^{-2}, 10^{1}]$ alongside VieClus and GNNS, while DMoN (NMI = 0.607 at $\sigma_c = 0.01$ and 0.474 at $d_{\text{out}} = 14$) and CommDGI (0.574 at $\sigma_c = 0.01$) provide a more balanced compromise between topology and attributes. (We omit LIM from Figure 5 as it frequently collapses to a single cluster on these synthetic instances.)

6 Conclusion

We presented a comprehensive empirical benchmark of 17 modularity-based graph clustering algorithms spanning exact mathematical programming, classical greedy and random-walk heuristics, streaming algorithms, parallel shared-memory heuristics, and recent deep learning architectures. When pure modularity maximization is the objective, classical combinatorial methods remain the clear state of the art: the evolutionary solver VieClus achieves the highest modularity on every instance it completes across both attributed and featureless benchmark suites, while Leiden matches VieClus within 1% modularity on the Ground-Truth graphs and executes more than four orders of magnitude faster.

At the same time, our multi-protocol evaluation clarifies where alternative paradigms excel and where fundamental bottlenecks remain:

- **Scalability vs. quality:** Single-pass streaming methods (CluStRE and Hollocou) process billion-edge graphs such as `uk-2007-05` (3.3 billion edges) within a few gigabytes of memory, and enabling multi-pass refinement in CluStRE closes nearly the entire

modularity gap to LEIDEN on medium-scale networks.

- **Ground-truth community recovery:** Because real-world semantic labels frequently diverge from purely topological modularity, attribute-aware neural methods (MAGI, DGCLUSTER, COMMDGI, and UCODE) match or outperform classical heuristics in NMI and Pairwise F1 on attributed networks. Moreover, our synthetic ADC-SBM experiments demonstrate that architectural choices—such as ego-feature concatenation in GraphSAGE versus Laplacian smoothing in GCNs—govern whether a neural method is resilient to structural noise or attribute noise.
- **Hardware acceleration and tuning sensitivity:** While GPU acceleration under Protocol S speeds up neural training by 12×–158× and enables full epoch convergence on medium-sized graphs, current GNN implementations remain constrained by $O(n^2)$ dense matrix allocations in host memory or the 24 GiB VRAM ceiling on GPUs. Furthermore, Protocol B demonstrates that whereas classical modularity heuristics work out of the box without tuning, multi-loss neural architectures often require careful hyperparameter optimization to avoid mode collapse, and still do not surpass untuned classical heuristics in raw modularity.

To support ongoing research, we release our open-source benchmarking harness, standardized dataset pipelines, and containerized solver wrappers. By decoupling algorithmic evaluation from ad-hoc preprocessing, cluster-count assumptions, and hardware disparities, we hope this framework establishes a rigorous foundation for developing the next generation of scalable, attribute-aware community detection methods.

Acknowledgements

We acknowledge support by DFG grant SCHU 2567/5-1.

Author Contributions

A.C., K.L., and C.S. conceived the study. K.L. and A.C. developed the benchmark framework, performed the experiments, and analyzed the data. K.L., A.C., and C.S. wrote the manuscript.

Competing Interests

The authors declare no competing interests.

Data and Code Availability

The complete benchmarking harness, solver wrappers, and datasets are open-source and publicly available at https://github.com/ClusterBenchmark/ClusterBenchmark, with interactive results hosted at https://clusterbenchmark.github.io/.

References

[1] Akoglu, L., Tong, H. & Koutra, D. Graph based anomaly detection and description: a survey. *Data Min. Knowl. Discov.* **29**, 626–688 (2015). URL https://doi.org/10.1007/s10618-014-0365-y.

[2] Cai, B., Wang, H., Zheng, H. & Wang, H. Detection of functional modules from protein interaction networks with an enhanced random walk based algorithm. *Int. J. Comput. Biol. Drug Des.* **4**, 290–306 (2011). URL https://doi.org/10.1504/IJCBDD.2011.041416.

[3] Xu, Y., Olman, V. & Xu, D. Clustering gene expression data using a graph-theoretic approach: an application of minimum spanning trees. *Bioinform.* **18**, 536–545 (2002). URL https://doi.org/10.1093/bioinformatics/18.4.536.

[4] Demme, J. & Sethumadhavan, S. Approximate graph clustering for program characterization. *ACM Trans. Archit. Code Optim.* **8**, 21:1–21:21 (2012). URL https://doi.org/10.1145/2086696.2086700.

[5] McFarling, S. Program optimization for instruction caches. In Emer, J. S. & Hennessy, J. L. (eds.) *ASPLOS-III Proceedings - Third International Conference on Architectural Support for Programming Languages and Operating Systems, Boston, Massachusetts, USA, April 3-6, 1989*, 183–191 (ACM Press, 1989). URL https://doi.org/10.1145/70082.68200.

[6] Newman, M. E. J. Properties of highly clustered networks. *Phys. Rev. E* **68**, 026121 (2003). URL https://link.aps.org/doi/10.1103/PhysRevE.68.026121.

[7] Peel, L., Larremore, D. B. & Clauset, A. The ground truth about metadata and community detection in networks. *CoRR* **abs/1608.05878** (2016). URL http://arxiv.org/abs/1608.05878. 1608.05878.

[8] Girvan, M. & Newman, M. E. J. Community structure in social and biological networks. *Proceedings of the National Academy of Sciences* **99**, 7821–7826 (2002). URL https://www.pnas.org/doi/abs/10.1073/pnas.122653799. https://www.pnas.org/doi/pdf/10.1073/pnas.122653799.

[9] Newman, M. E. J. Modularity and community structure in networks. *Proceedings of the National Academy of Sciences* **103**, 8577–8582 (2006). URL https://www.pnas.org/doi/abs/10.1073/pnas.0601602103. https://www.pnas.org/doi/pdf/10.1073/pnas.0601602103.

[10] Bader, D. A., Meyerhenke, H., Sanders, P. & Wagner, D. (eds.). *Graph Partitioning and Graph Clustering, 10th DIMACS Implementation Challenge Workshop, Georgia Institute of Technology, Atlanta, GA, USA, February 13-14, 2012. Proceedings*, vol. 588 of *Contemporary Mathematics* (American Mathematical Society, 2013). URL https://doi.org/10.1090/conm/588.

[11] Li, J. *et al.* A Comprehensive Review of Community Detection in Graphs. *Neurocomputing* **600**, 128169 (2024).

[12] Blöcker, C., Tan, C. & Scholtes, I. The map equation goes neural: Mapping network flows with graph neural networks. In *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS 2024)*, 17428–17456 (Curran Associates, Inc., 2024).

[13] Wani, A. A. Comprehensive analysis of clustering algorithms: exploring limitations and innovative solutions. *PeerJ Comput. Sci.* **10**, e2286 (2024). URL https://doi.org/10.7717/peerj-cs.2286.

[14] Liu, F. *et al.* Deep learning for community detection: Progress, challenges and opportunities. In Bessiere, C. (ed.) *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI 2020*, 4981–4987 (ijcai.org, 2020). URL https://doi.org/10.24963/ijcai.2020/693.

[15] Jin, D. *et al.* A survey of community detection approaches: From statistical modeling to deep learning. *IEEE Trans. Knowl. Data Eng.* **35**, 1149–1170 (2023). URL https://doi.org/10.1109/TKDE.2021.3104155.

[16] Aref, S., Mostajabdaveh, M. & Chheda, H. Heuristic modularity maximization algorithms for community detection rarely return an optimal partition or anything similar. In *Proceedings of the 23th International Conference on Computational Science (ICCS 2023)*, 612–626 (Springer Nature Switzerland, 2023).

[17] Li, J. *et al.* A comprehensive review of community detection in graphs. *Neurocomputing* **600**, 128169 (2024). URL https://doi.org/10.1016/j.neucom.2024.128169.

[18] Liang, K. *et al.* A survey of knowledge graph reasoning on graph types: Static, dynamic, and multi-modal. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **46**, 9456–9478 (2024).

[19] Messias, G. H., Gonçalves Botelho, T., Iasulaitis, S., Coelho Naldi, M. & Demétrius Baria Valejo, A. A systematic review of graph neural network for node clustering in homogeneous attributed networks. *IEEE Access* **13**, 215265–215285 (2025).

[20] Džamić, D., Aloise, D. & Mladenović, N. Ascent-descent variable neighborhood decomposition search for community detection by modularity maximization. *Annals of Operations Research* **272** (2019).

[21] Aloise, D. *et al.* Modularity maximization in networks by variable neighborhood search. In *Graph Partitioning and Graph Clustering*, vol. 588 of *Contemporary Mathematics*, 113–127 (American Mathematical Society, 2012).

[22] Ovelgönne, M. & Geyer-Schulz, A. An ensemble learning strategy for graph clustering. In *Graph Partitioning and Graph Clustering*, vol. 588 of *Contemporary Mathematics*, 187–205 (American Mathematical Society, 2012).

[23] Biedermann, S., Henzinger, M., Schulz, C. & Schuster, B. Memetic graph clustering. In *Proceedings of the 17th International Symposium on Experimental Algorithms (SEA 2018)*, 3:1–3:15 (Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018).

[24] Ansótegui, C., Ramaswamy, V. P., Szeider, S. & Xia, H. Uncovering and verifying optimal community structure in complex networks: A MaxSAT approach. In *Proceedings of the 25th International Conference on Computational Science (ICCS 2025)*, 35–49 (Springer Nature Switzerland, 2025).

[25] Chhabra, A., Dorian Peretz, S. & Schulz, C. CluStRE: Streaming graph clustering with multi-stage refinement. In *Proceedings of the 23rd International Symposium on Experimental Algorithms (SEA 2025)*, 11:1–11:20 (Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025).

[26] Sobolevsky, S. & Belyi, A. Graph neural network inspired algorithm for unsupervised network community detection. *Applied Network Science* **7**, 63 (2022).

[27] Sobolevsky, S., Campari, R., Belyi, A. & Ratti, C. General optimization technique for high-quality community detection in complex networks. *Physical Review E* **90**, 012811 (2014).

[28] Pons, P. & Latapy, M. Computing communities in large networks using random walks. In *Proceedings of the 20th International Symposium on Computer and Information Sciences (ISCIS 2005)*, 284–293 (Springer Berlin Heidelberg, 2005).

[29] Hollocou, A., Maudet, J., Bonald, T. & Lelarge, M. A streaming algorithm for graph clustering. arXiv preprint (2017).

[30] Traag, V. A., Waltman, L. & van Eck, N. J. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports* **9**, 5233 (2019).

[31] Aref, S., Mostajabdaveh, M. & Chheda, H. Bayan algorithm: Detecting communities in networks through exact and approximate optimization of modularity. *Physical Review E* **110**, 044315 (2024).

[32] Blondel, V. D., Guillaume, J.-L., Lambiotte, R. & Lefebvre, E. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment* **2008**, P10008 (2008).

[33] Clauset, A., Newman, M. E. J. & Moore, C. Finding community structure in very large networks. *Physical Review E* **70**, 066111 (2004).

[34] Wakita, K. & Tsurumi, T. Finding community structure in mega-scale social networks: [extended abstract]. In *Proceedings of the 16th International Conference on World Wide Web (WWW 2007)*, 1275–1276 (Association for Computing Machinery, 2007).

[35] Wang, H. *et al.* Simple yet effective heuristic community detection with graph convolution network. *Scientific Reports* **15**, 39249 (2025).

[36] Zhang, T. *et al.* CommDGI: Community detection oriented deep graph infomax. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management (CIKM 2020)*, 1843–1852 (Association for Computing Machinery, 2020).

[37] Moradan, A., Draganov, A., Mottin, D. & Assent, I. UCoDe: Unified community detection with graph convolutional networks. *Machine Learning* **112**, 5057–5080 (2023).

[38] Bhowmick, A., Kosan, M., Huang, Z., Singh, A. & Medya, S. DGCLUSTER: A neural framework for attributed graph clustering via modularity maximization. In *Proceedings of the 38th AAAI Conference on Artificial Intelligence (AAAI 2024)*, 11069–11077 (AAAI Press, Washington, DC, USA, 2024).

[39] Liu, Y. *et al.* Revisiting modularity maximization for graph clustering: A contrastive learning perspective. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD 2024)*, 1968–1979 (Association for Computing Machinery, New York, NY, United States, 2024).

[40] Tsitsulin, A., Palowitch, J., Perozzi, B. & Müller, E. Graph clustering with graph neural networks. *Journal of Machine Learning Research* **24**, 1–21 (2023). URL http://jmlr.org/papers/v24/20-998.html.

[41] Gaertler, M. Clustering. In Brandes, U. & Erlebach, T. (eds.) *Network Analysis: Methodological Foundations [outcome of a Dagstuhl seminar, 13-16 April 2004]*, vol. 3418 of *Lecture Notes in Computer Science*, 178–215 (Springer, 2004). URL https://doi.org/10.1007/978-3-540-31955-9_8.

[42] Kannan, R., Vempala, S. S. & Vetta, A. On clusterings: Good, bad and spectral. *J. ACM* **51**, 497–515 (2004). URL https://doi.org/10.1145/990308.990313.

[43] Arnau, V., Mars, S. & Marín, I. Iterative Cluster Analysis of Protein Interaction Data. *Bioinformatics* **21**, 364–378 (2004).

[44] Prat-Pérez, A., Dominguez-Sal, D., Brunat, J. M. & Larriba-Pey, J.-L. Shaping communities out of triangles. In *Proceedings of the 21st ACM international conference on Information and knowledge management*, 1677–1681 (Association for Computing Machinery, 2012).

[45] Prat-Pérez, A., Dominguez-Sal, D. & Larriba-Pey, J.-L. High quality, scalable and parallel community detection for large real graphs. In *Proceedings of the 23rd International Conference on World Wide Web (WWW 2014)*, 225–236 (Association for Computing Machinery, 2014).

[46] Rosvall, M. & Bergstrom, C. T. Maps of random walks on complex networks reveal community structure. *Proceedings of the National Academy of Sciences* **105**, 1118–1123 (2008).

[47] Fung, P. K. InfoFlow: A distributed algorithm to detect communities according to the map equation. *Big Data and Cognitive Computing* **3**, 42 (2019).

[48] Rossetti, G., Milli, L. & Cazabet, R. CDLIB: a python library to extract, compare and evaluate communities from complex networks. *Applied Network Science* **4**, 2364–8228 (2019).

[49] Csardi, G. & Nepusz, T. The igraph software package for complex network research. *InterJournal* **Complex Systems**, 1695 (2006). URL https://igraph.org.

[50] Kats, P. & Belyi, A. COMBO repository. https://github.com/Casyfill/pyCombo.git (2026).

[51] Hollocou, A. Hollocou repository. https://github.com/ahollocou/graph-streaming.git (2026).

[52] Schulz, C. & Seemaier, D. VieClus repository. https://github.com/VieClus/VieClus.git (2026).

[53] Tsitsulin, A., Palowitch, J., Perozzi, B. & Müller, E. DMoN repository. https://github.com/google-research/google-research.git (2026).

[54] Zhang, T. *et al.* CommDGI repository. https://github.com/FDUDSDE/CommDGI.git (2026).

[55] Belyi, A. GNNS repository. https://github.com/Alexander-Belyi/GNNS.git (2026).

[56] Moradan, A., Draganov, A., Mottin, D. & Assent, I. UCoDe repository. https://github.com/AU-DIS/UCODE.git (2026).

[57] Kosan, M. DGCluster repository. https://github.com/pyrobits/DGCluster.git (2026).

[58] Li, J. MAGI repository. https://github.com/EdisonLeeeee/MAGI.git (2026).

[59] Mostajabdaveh, M. & Aref, S. Bayan repository. https://github.com/saref/bayan.git (2026).

[60] Chhabra, A. CluStRE repository. https://github.com/KaHIP/CluStRE.git (2026).

[61] Wang, H. LIM repository. https://github.com/wuanghoong/Less-is-More.git (2026).

[62] Brandes, U. *et al.* On modularity clustering. *IEEE Trans. Knowl. Data Eng.* **20**, 172–188 (2008). URL https://doi.org/10.1109/TKDE.2007.190689.

[63] Zhang, T., Ramakrishnan, R. & Livny, M. Birch: an efficient data clustering method for very large databases. In *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*, SIGMOD '96, 103–114 (Association for Computing Machinery, New York, NY, USA, 1996). URL https://doi.org/10.1145/233269.233324.

[64] Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual (2026). URL https://www.gurobi.com.

[65] Tsitsulin, A., Rozemberczki, B., Palowitch, J. & Perozzi, B. Synthetic graph generation to benchmark graph learning. arXiv preprint (2022).

[66] Danon, L., Diaz-Guilera, A., Duch, J. & Arenas, A. Comparing community structure identification. *Journal of Statistical Mechanics: Theory and Experiment* **2005**, 2364–8228 (2005).

[67] Dolan, E. D. & Moré, J. J. Benchmarking optimization software with performance profiles. *Mathematical Programming* **91**, 201–213 (2002).

[68] Akiba, T., Sano, S., Yanase, T., Ohta, T. & Koyama, M. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2623–2631 (2019).

A Benchmark Instances

Table 5: Graphs from the 10th DIMACS Implementation Challenge [10] used in our topology-only evaluation suite ($N = 32$). These instances do not include vertex features or ground-truth community labels.

Instance	n	m	Instance	n	m
karate	34	78	astro-ph	16 706	121 251
chesapeake	39	170	cond-mat	16 726	47 594
dolphins	62	159	as-22july06	22 963	48 436
lesmis	77	254	cond-mat-2003	31 163	120 029
polbooks	105	441	cond-mat-2005	40 421	175 691
adjnoun	112	425	preferentialAttachment	100 000	499 985
football	115	613	smallworld	100 000	499 998
jazz	198	2 742	G-n-pin-pout	100 000	501 198
celegansneural	297	2 148	caidaRouterLevel	192 244	609 066
celegans-metabolic	453	2 025	cnr-2000	325 557	2 738 969
email	1 133	5 451	eu-2005	862 664	16 138 468
polblogs	1 490	16 715	in-2004	1 382 908	13 591 473
netscience	1 589	2 742	road-central	14 081 816	16 933 413
power	4 941	6 594	uk-2002	18 520 486	261 787 258
hep-th	8 361	15 751	road-usa	23 947 347	28 854 312
PGPgiantcompo	10 680	24 316	uk-2007-05	105 896 555	3 301 876 564

B Memory Usage and Training Convergence

Table 6: Peak memory consumption of the evaluated algorithms on the Ground-Truth dataset under Protocol A (recorded via Linux `cgroup` tracking). Entries marked TLE indicate runs exceeding the one-hour time limit, while MLE denotes runs exceeding the 120 GB memory limit.

Method	Cora	Citeseer	Photo	Comp	Co-CS	Pubmed	Co-Phys	Arxiv	Reddit	Prod
VieClus	30.5 MB	30.2 MB	74.8 MB	162 MB	67.9 MB	53.9 MB	146 MB	506 MB	11.4 GB	TLE
Leiden	21.2 MB	20.8 MB	43.4 MB	85.2 MB	36.2 MB	29.0 MB	71.4 MB	265 MB	11.7 GB	12.7 GB
Louvain	20.5 MB	20.1 MB	43.5 MB	80.7 MB	35.9 MB	29.5 MB	71.5 MB	265 MB	11.7 GB	12.7 GB
COMBO	152 MB	212 MB	1.01 GB	3.09 GB	5.33 GB	TLE	TLE	MLE	MLE	MLE
CNM	21.0 MB	21.4 MB	43.4 MB	73.7 MB	39.7 MB	40.6 MB	71.3 MB	264 MB	TLE	TLE
Infomap	33.7 MB	36.3 MB	103 MB	211 MB	138 MB	118 MB	288 MB	1.28 GB	25.8 GB	38.5 GB
Walktrap	33.9 MB	25.2 MB	434 MB	1.49 GB	969 MB	531 MB	4.38 GB	99.3 GB	TLE	MLE
CluStRE	17.8 MB	17.6 MB	20.4 MB	38.8 MB	19.9 MB	20.1 MB	25.2 MB	56.2 MB	986 MB	1.10 GB
Hollocou	8.9 MB	8.9 MB	11.2 MB	14.4 MB	12.3 MB	13.0 MB	16.7 MB	43.2 MB	942 MB	1.14 GB
GNNS	719 MB	784 MB	1.49 GB	2.59 GB	2.75 GB	2.91 GB	4.94 GB	8.74 GB	40.3 GB	46.5 GB
DGCluster	520 MB	540 MB	1.06 GB	2.22 GB	1.08 GB	1.13 GB	1.93 GB	6.10 GB	MLE	MLE
MAGI	790 MB	945 MB	1.56 GB	2.99 GB	5.78 GB	2.13 GB	13.8 GB	7.05 GB	43.3 GB	TLE
LIM	676 MB	786 MB	2.14 GB	5.02 GB	6.57 GB	6.80 GB	21.0 GB	MLE	MLE	MLE
DMoN	436 MB	524 MB	1.46 GB	3.16 GB	2.54 GB	1.20 GB	5.82 GB	14.0 GB	MLE	MLE
UCoDe	614 MB	718 MB	1.75 GB	4.83 GB	6.79 GB	6.93 GB	22.1 GB	MLE	MLE	MLE
CommDGI	735 MB	783 MB	1.91 GB	5.55 GB	8.91 GB	9.73 GB	29.7 GB	MLE	MLE	MLE

Table 7: Number of completed training epochs/iterations out of the requested budget on the GROUND-TRUTH dataset under Protocol A (single-threaded CPU, 1-hour limit). Note that GNNS employs early stopping upon convergence.

Method	Cora	Citeseer	Photo	Comp	Co-CS	Pubmed	Co-Phys	Arxiv	Reddit	Prod
GNNS	50/400	50/400	50/400	50/400	50/400	50/400	50/400	400/400	120/400	300/300
DGCLUSTER	300/300	300/300	300/300	300/300	300/300	300/300	300/300	270/300	MLE	MLE
MAGI	400/400	400/400	400/400	142/400	50/400	140/400	12/400	5/400	7/400	TLE
LIM	300/300	300/300	300/300	300/300	300/300	300/300	111/300	MLE	MLE	MLE
DMoN	1000/1000	1000/1000	1000/1000	1000/1000	1000/1000	1000/1000	819/1000	583/1000	MLE	MLE
UCoDe	1000/1000	1000/1000	1000/1000	1000/1000	960/1000	1000/1000	280/1000	MLE	MLE	MLE
CommDGI	1001/1001	1001/1001	1001/1001	986/1001	564/1001	560/1001	166/1001	MLE	MLE	MLE

C Hardware Scaling Comparison (Protocol S)

Table 8: Execution time in seconds across hardware configurations on six medium-scale GROUND-TRUTH instances comparing Protocol A (t_1: 1 CPU thread) against Protocol S (t_{32}: 32 CPU threads; t_{GPU}: 32 CPU threads + NVIDIA RTX 4090 GPU). Note that VIECLUS is a CPU-only MPI solver with a 1-hour time budget.

	Cora			**Citeseer**			**Amz-Photo**			**Amz-Comp**			**CoAuth-CS**			**CoAuth-Phys**		
Method	t_1	t_{32}	t_{GPU}	t_1	t_{32}	t_{GPU}	t_1	t_{32}	t_{GPU}	t_1	t_{32}	t_{GPU}	t_1	t_{32}	t_{GPU}	t_1	t_{32}	t_{GPU}
VIECLUS	3456	3457	—	3456	3457	—	3456	3458	—	3457	3458	—	3456	3457	—	3457	3458	—
GNNS	5.3	4.6	0.29	6.7	5.9	0.36	19.9	20.7	1.10	45.1	51.0	2.68	48.4	65.5	2.97	99.0	174.7	8.10
DGCLUSTER	25.6	6.7	2.47	43.4	8.2	2.14	287	68.2	5.98	604	132	9.69	494	74.8	7.51	1266	187	17.3
MAGI	162	85.8	58.2	301	107	61.4	3015	1442	538	3606	3615	1554	3616	3623	2314	3646	3657	3654
LIM	66.4	7.0	0.97	140	17.5	1.08	695	104	3.69	1877	239	9.12	2960	303	11.7	3615	912	40.5
DMoN	93.8	36.7	18.8	197	43.5	19.4	480	150	49.8	971	310	91.1	1862	253	45.9	3604	653	124
UCoDe	91.4	12.3	1.81	184	20.4	1.88	559	58.2	3.81	1740	196	11.6	3605	401	23.6	3616	1480	78.0
CommDGI	107	33.6	3.25	258	69.9	3.29	1217	280	7.48	3603	727	19.9	3605	1123	36.3	3625	3525	MLE